

Project Title:

Threat Modeling and Security Analysis of OWASP Juice Shop

Mahrukh

TABLE OF CONTENTS

1. Introduction
 - 1.1 Project Overview
 - 1.2 Objectives
 - 1.3 Selected Application
2. System Architecture
 - 2.1 Application Description
 - 2.2 System Components
 - 2.3 Data Flow Diagram (DFD)
3. Threat Modeling (STRIDE)
 - 3.1 STRIDE Overview
 - 3.2 Identified Threats
4. Threat Validation
 1. Cross-Site Scripting
 2. SQL Injection
 3. No Rate limiting
 4. Sensitive Data Exposure
 5. Broken Access Control
 6. Weak Password Policy
5. Risk Assessment
6. Mitigation and Recommendations
7. Conclusion

1. INTRODUCTION**1.1 Project Overview**

Web applications are widely used in modern systems but often contain vulnerabilities that can be exploited by attackers. This project focuses on analyzing security weaknesses in a web application using structured threat modeling techniques.

1.2 Objectives

The main objectives of this project are:

- To understand the application architecture
- To create a Data Flow Diagram (DFD)
- To identify threats using STRIDE methodology
- To validate vulnerabilities through testing
- To perform risk analysis
- To recommend mitigation strategies

1.3 Selected Application

OWASP Juice Shop is an intentionally vulnerable web application designed for security training. It simulates an e-commerce platform and contains vulnerabilities such as SQL Injection, Cross-Site Scripting (XSS), and Broken Authentication.

2. SYSTEM ARCHITECTURE

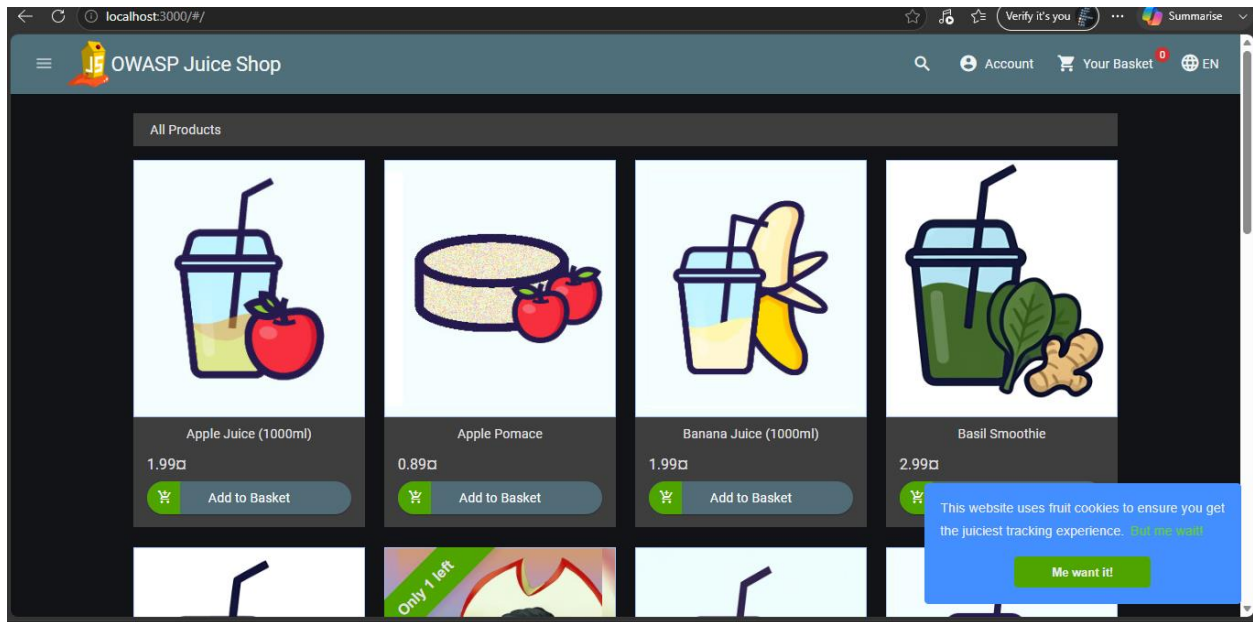
2.1 Application Description

The application is a web-based e-commerce system where users can browse products, log in, and perform various actions. It follows a client-server architecture.

```
Microsoft Windows [Version 10.0.26200.8655]
(c) Microsoft Corporation. All rights reserved.

C:\Users\HP>docker --version
Docker version 29.5.3, build d1c06ef

C:\Users\HP>docker run -d -p 3000:3000 bkimminich/juice-shop
Unable to find image 'bkimminich/juice-shop:latest' locally
latest: Pulling from bkimminich/juice-shop
6b2d9fb43c80: Downloading [=====>] 6.291MB/61.43MB
dd0d9bfd3bee: Pull complete
ebddc55facdc: Pull complete
702a7246bd56: Downloading [==>] 2.097MB/44.08MB
1a4be5562d92: Pull complete
47de5dd0b812: Pull complete
8224d91da70b: Pull complete
99515e7b4d35: Pull complete
7c12895b777b: Pull complete
7cc70dfd88cf: Pull complete
bdfd77fe5bf6: Pull complete
2d4d7ad6272: Pull complete
bd8962e29291: Pull complete
52630fc75a18: Pull complete
c172f21841df: Pull complete
cf397222899a: Pull complete
48fce22aed6d: Download complete
3214acf345c0: Pull complete
cac2ae0193cb: Pull complete
dd64bf2dd177: Pull complete
b839d4ae01f6: Pull complete
d6b1b89eccac: Pull complete
99ba982a9142: Pull complete
2780920e5dbf: Pull complete
8cd642a6e056: Download complete
```



2.2 System Components

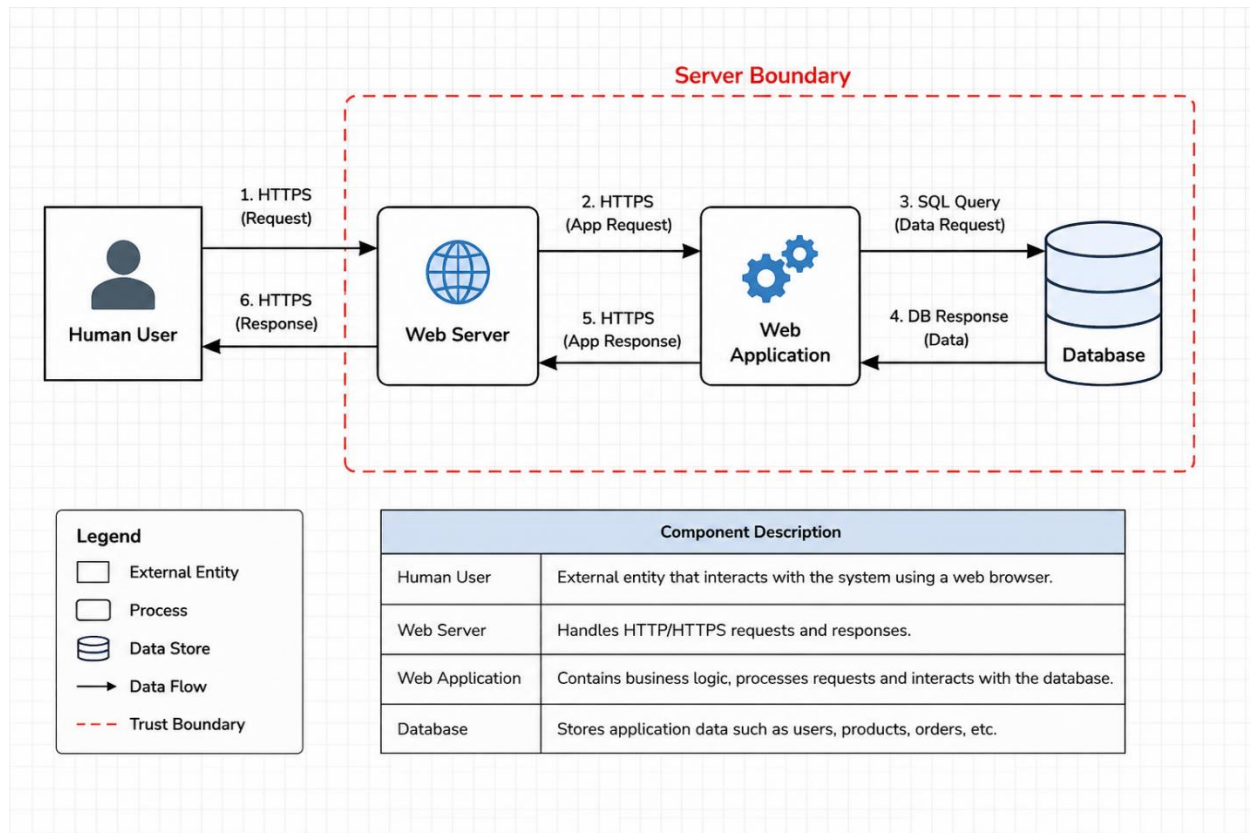
The system consists of the following components:

- **User (External Entity):** Interacts through a web browser
- **Web Server:** Handles HTTP/HTTPS requests
- **Web Application:** Processes business logic
- **Database:** Stores user and product data

2.3 Data Flow Diagram (DFD)

The DFD represents how data flows within the system:

- User sends request → Web Server
- Web Server forwards → Application
- Application interacts → Database
- Response returned → User



3. THREAT MODELING (STRIDE)

3.1 STRIDE Overview

STRIDE is a threat modeling technique used to identify security risks:

- Spoofing
- Tampering
- Repudiation
- Information Disclosure
- Denial of Service
- Elevation of Privilege

3.2 Identified Threats

1. Spoofing (Weak Authentication)

Users can create accounts with weak passwords such as "123", showing poor authentication controls.

2. Tampering (XSS / Input Manipulation)

User input fields allow injection of malicious scripts.

3. Information Disclosure

Application reveals internal errors and system details.

4. Elevation of Privilege

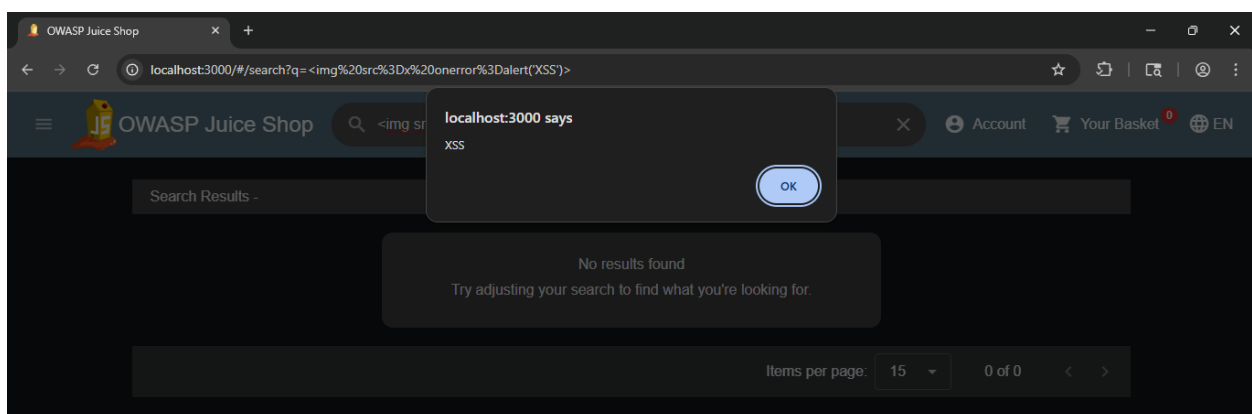
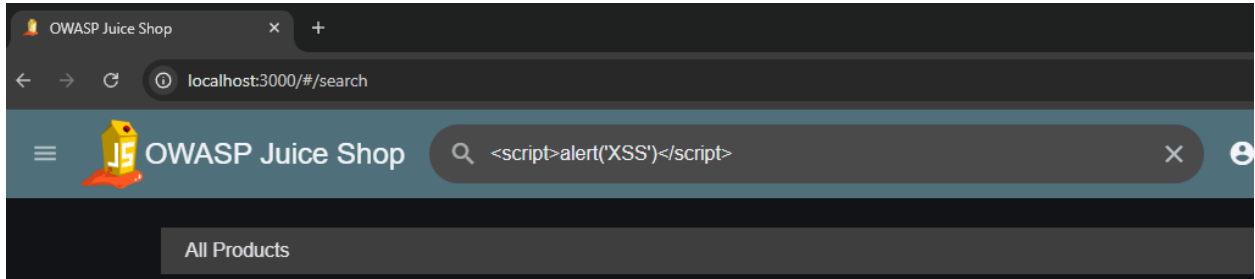
Hidden pages like Score Board can be accessed without authorization.

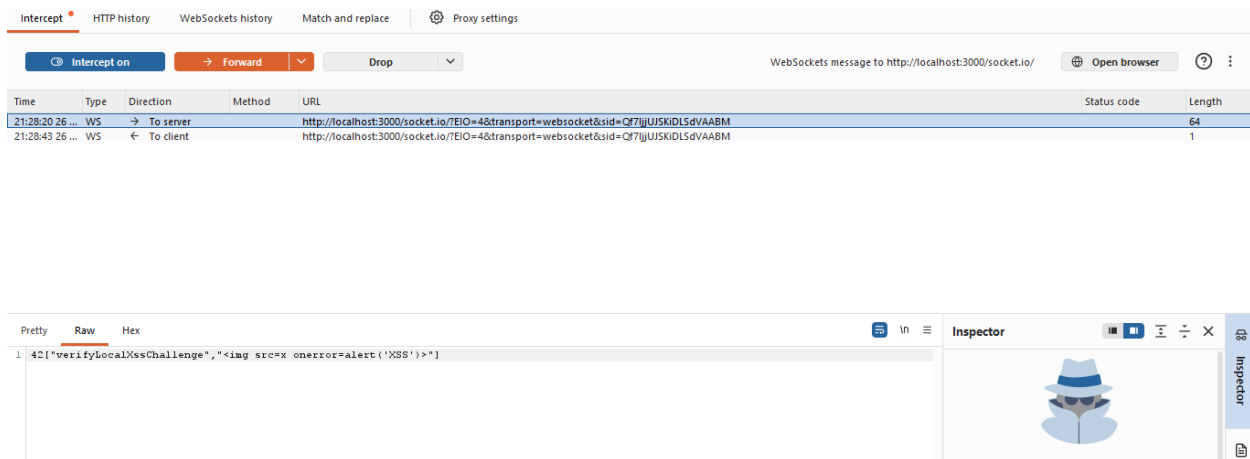
4. THREAT VALIDATION

Threat 1: Cross-Site Scripting (XSS)

Description

- **Type:** Code-level vulnerability (Tampering).
- **Location:** User comment fields.
- **Cause:** Inadequate context-aware output encoding allowed malicious JavaScript injection.
- **Impact:** Attackers could execute arbitrary scripts in the victim's browser, leading to session hijacking, credential theft, or defacement.





Fix Implementation

- **Frontend (Angular):**
 - Replaced unsafe rendering with Angular's automatic escaping (`{{ comment.text }}` instead of `innerHTML`).
- **Backend (Node.js/Express):**
 - Added middleware (xss-clean) to sanitize incoming comment data.
 - Implemented server-side validation to reject suspicious payloads containing `<script>` tags or event handlers.

Testing with Burp Suite

Testing was conducted using **Burp Suite** to intercept and modify HTTP requests, ensuring that malicious payloads no longer executed in the browser.

1. Script Injection Test

- Payload: `<script>alert('XSS')</script>`
- **Before Fix:** Alert popup executed in browser.
- **After Fix:** Payload appeared only as escaped text (`<script>alert('XSS')</script>`) in Burp response and browser view.

2. Event Handler Injection Test

- Payload: `<img src=x onerror=alert('XSS')>`
- **Before Fix:** Alert triggered via image error.

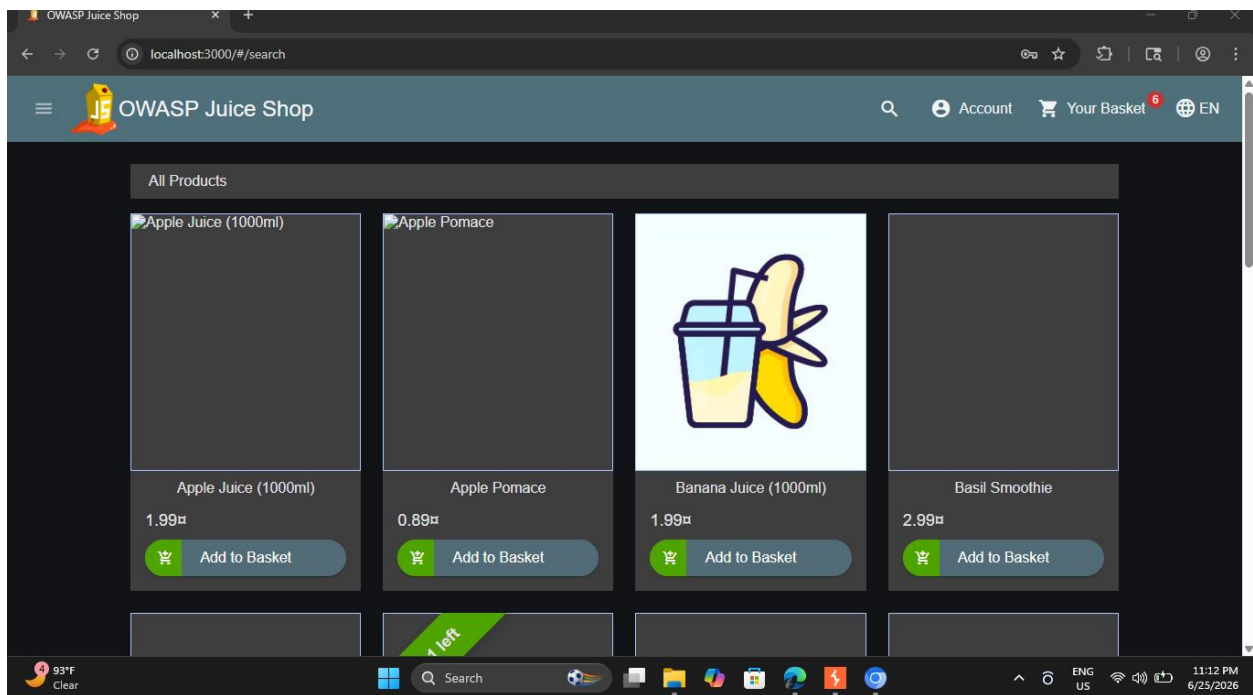
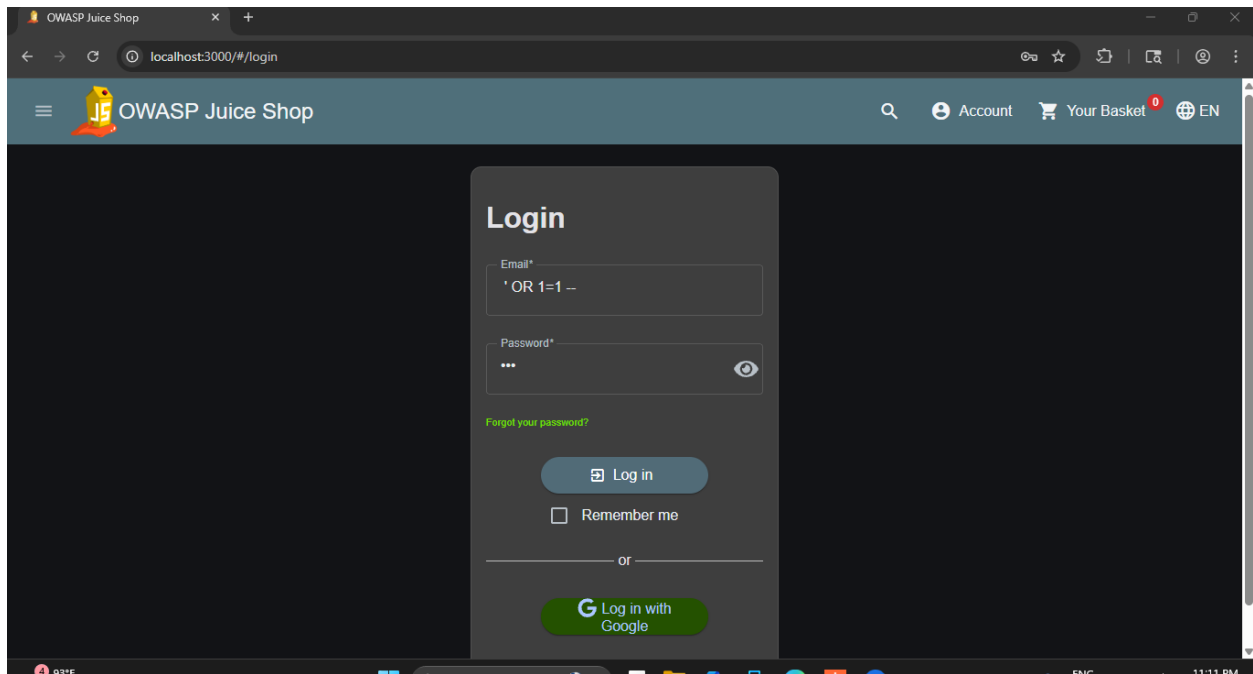
- **After Fix:** Payload sanitized, displayed harmlessly as text. Burp confirmed no script execution.

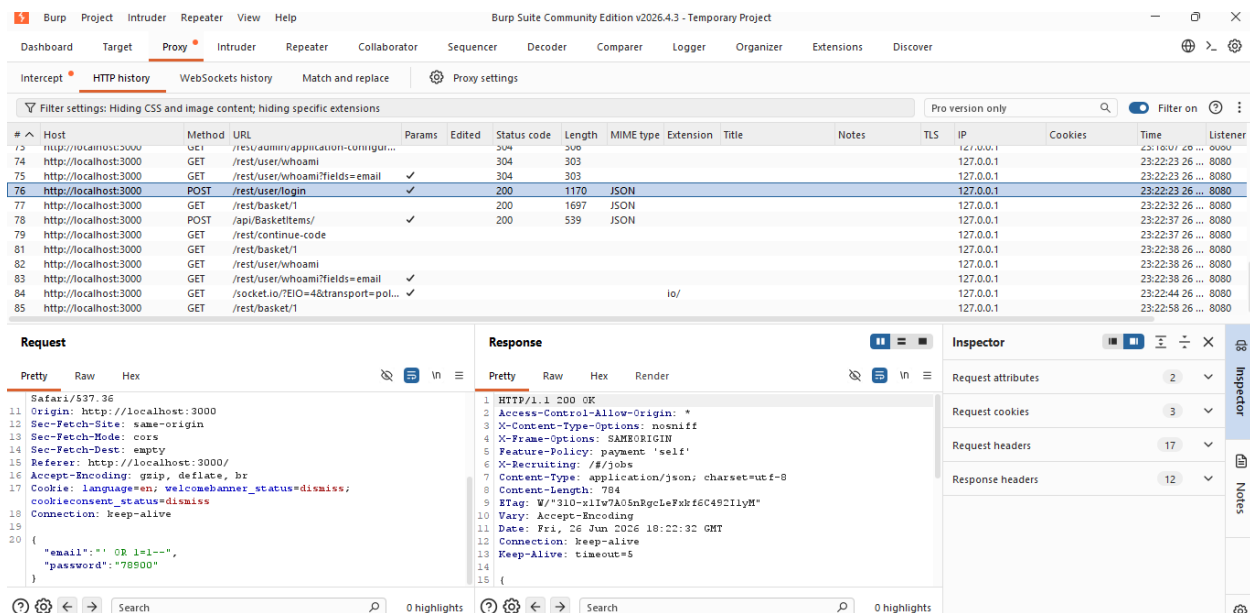


Threat 2: SQL Injection (Authentication Bypass)

Description

- **Type:** Code-level vulnerability (Injection).
- **Location:** Login authentication endpoint (/rest/user/login).
- **Cause:** User input was directly used in database query without proper parameterization or sanitization, allowing SQL logic manipulation.
- **Impact:** Attackers could bypass authentication using malicious input (e.g., ' OR 1=1--), leading to unauthorized access and account takeover.





Fix Implementation

• Backend (Node.js/Express):

- o Replaced unsafe string-based query handling with secure parameterized/ORM-based query logic.
- o Ensured all user inputs are treated strictly as data, not executable SQL commands.
- o Strengthened authentication logic to validate credentials properly before granting access.

• Frontend (Angular):

- o Added basic input validation on login form to prevent malformed input submission.

```

    next(error)
  })
}

return (req: Request, res: Response, next: NextFunction) => {
  verifyPreLoginChallenges(req) // vuln-code-snippet hide-line
  models.sequelize.query(
    "SELECT * FROM Users WHERE email = ? AND password = ? AND deletedAt IS NULL",
    {
      replacements: [
        req.body.email || '',
        security.hash(req.body.password || '')
      ],
      model: UserModel,
      plain: true
    }
  ) // vuln-code-snippet vuln-line loginAdminChallenge loginBenderChallenge loginJimChallenge
    .then((authenticatedUser) => { // vuln-code-snippet neutral-line loginAdminChallenge loginBenderChallenge loginJimChallenge
      const user = utils.queryResultToJson(authenticatedUser)
    })
}

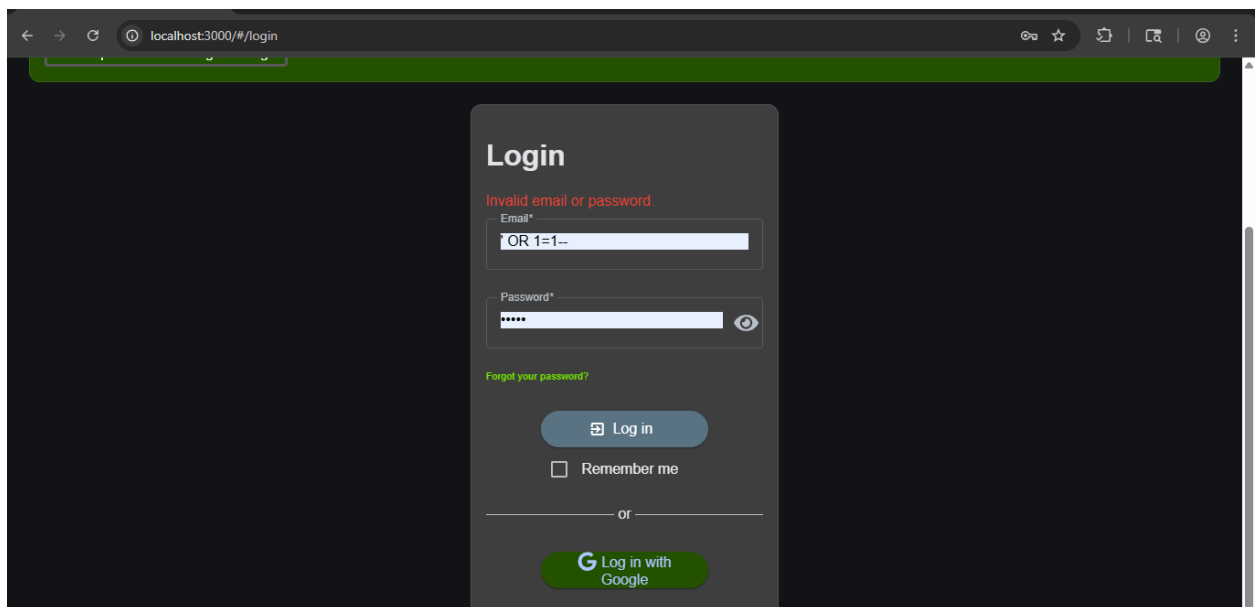
```

Testing (Web + Burp Suite Verification)

Testing was performed in two stages to confirm the vulnerability and validate the fix.

1. Web Application Testing:

- o First, the attack was tested directly on the Juice Shop login page using SQL injection payloads such as ' OR 1=1--.
- o Before the fix, this allowed login bypass and unauthorized access.
- o After applying the fix, the same payload was tested again and login was correctly rejected.



2. Burp Suite Verification:

- o After web testing, Burp Suite was used to intercept and modify the login request.
- o Malicious payloads were injected into the request body to test backend validation.
- o The server response confirmed that injected SQL payloads were treated as normal input and did not affect authentication.

45	http://localhost:3000	GET	/rest/admin/application-configur...	304	300		127.0.0.1
56	http://localhost:3000	GET	/rest/saveLoginIp	200	730	JSON	127.0.0.1
57	http://localhost:3000	GET	/rest/admin/application-configur...	304	306		127.0.0.1
58	http://localhost:3000	POST	/rest/user/login	401	413	HTML	127.0.0.1
59	http://localhost:3000	GET	/rest/user/whoami	200	394	JSON	127.0.0.1
60	http://localhost:3000	GET	/rest/user/whoami?fields=email	200	394	JSON	127.0.0.1
61	http://localhost:3000	GET	/rest/user/whoami?fields=email	304	303		127.0.0.1
62	http://localhost:3000	POST	/rest/user/login	401	413	HTML	127.0.0.1
63	http://localhost:3000	GET	/rest/user/whoami	304	303		127.0.0.1

Request				Response			
Pretty	Raw	Hex		Pretty	Raw	Hex	Render
<pre> 11 Origin: http://localhost:3000 12 Sec-Fetch-Site: same-origin 13 Sec-Fetch-Mode: cors 14 Sec-Fetch-Dest: empty 15 Referer: http://localhost:3000/ 16 Accept-Encoding: gzip, deflate, br 17 Cookie: language=en; welcomebanner_status=dismiss; 18 cookieconsent_status=dismiss; continueCode= 19 wjKRWbLkRo7kYs8mJyQ3p5aJEDngB0HVVU5evLWwPNxZq64KV20rMDGb1og1LX 20 Connection: keep-alive { "email": "", OR 1=--, "password": "78900" } </pre>				<pre> 1 HTTP/1.1 401 Unauthorized 2 Access-Control-Allow-Origin: * 3 X-Content-Type-Options: nosniff 4 X-Frame-Options: SAMEORIGIN 5 Feature-Policy: payment 'self' 6 X-Recruiting: /#/jobs 7 Content-Type: text/html; charset=utf-8 8 Content-Length: 26 9 ETag: W/"1a-ARJvVK+smzAF3QQue2mDSG+3Bus" 10 Vary: Accept-Encoding 11 Date: Sat, 27 Jun 2026 14:53:20 GMT 12 Connection: keep-alive 13 Keep-Alive: timeout=5 14 15 Invalid email or password. </pre>			

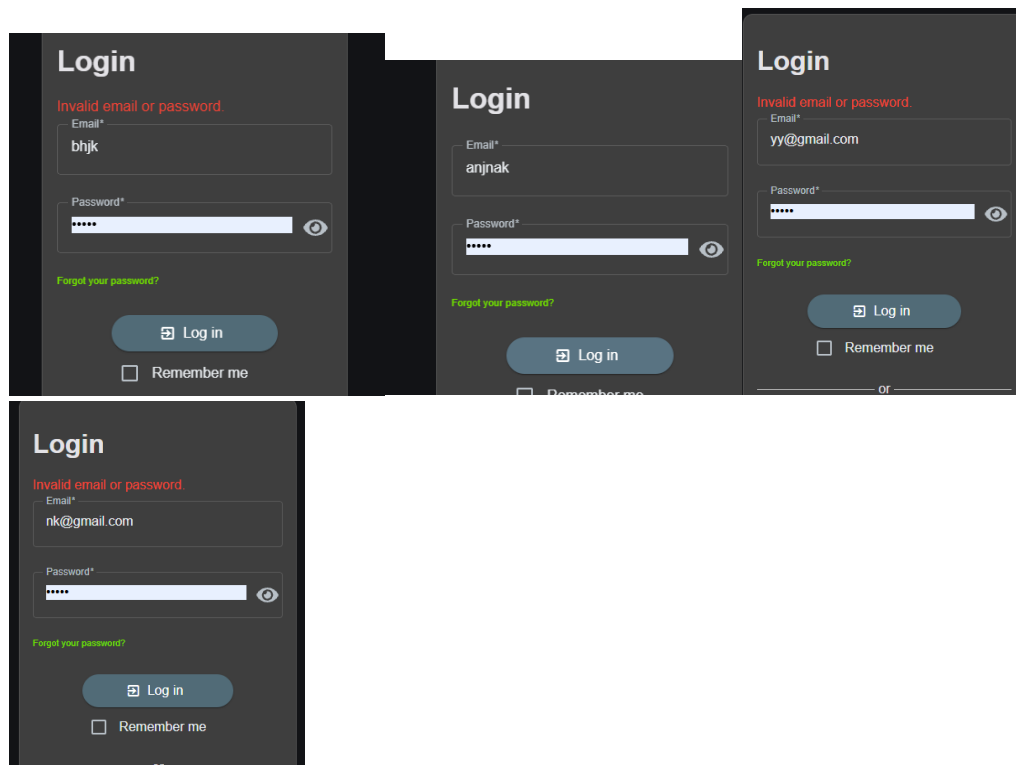
Result

The SQL injection vulnerability was successfully mitigated by implementing secure parameterized authentication logic in the backend and adding input validation in the frontend. Both direct web testing and Burp Suite interception confirmed that SQL injection attempts no longer bypass authentication or manipulate queries.

Threat 3: No Rate Limiting (Brute Force Attack)

Description

- **Type:** Code-level vulnerability (Authentication/Brute Force).
- **Location:** Login endpoint (/rest/user/login).
- **Cause:** The application does not restrict repeated login attempts, allowing unlimited password guesses without any delay or blocking mechanism.
- **Impact:** Attackers can perform brute force attacks by repeatedly trying different password combinations, potentially leading to unauthorized account access.



Fix Implementation

- **Backend (Node.js/Express):**

- o Implemented rate limiting middleware using express-rate-limit.
- o Restricted the number of login attempts within a defined time window.
- o Blocked excessive requests after the limit is exceeded.

- **Configuration Applied:**

- o Maximum attempts: 5 requests
- o Time window: 1 minute
- o Block response: HTTP 429 (Too Many Requests)

```
File Edit View
import * as utils from '../lib/utils'

// vuln-code-snippet start loginAdminChallenge loginBenderChallenge loginJimChallenge
export function login () {
  function afterLogin (user: User, res: Response, next: NextFunction) {
    verifyPostLoginChallenges(user) // vuln-code-snippet hide-line
    BasketModel.findOrCreate({ where: { UserId: user.id } })
      .then(([basket]: [BasketModel, boolean]) => {
        const authenticatedUser = { data: user, bid: basket.id } // keep track of original basket
        const token = security.authorize(authenticatedUser)
        security.authenticatedUsers.put(token, authenticatedUser)
        res.json({ authentication: { token, bid: basket.id, uemail: user.email } })
      }).catch((error: Error) => {
        next(error)
      })
  }

  return (req: Request, res: Response, next: NextFunction) => {
    verifyPreLoginChallenges(req) // vuln-code-snippet hide-line
    models.sequelize.query(
      "SELECT * FROM Users WHERE email = ? AND password = ? AND deletedAt IS NULL",
      {
        replacements: [

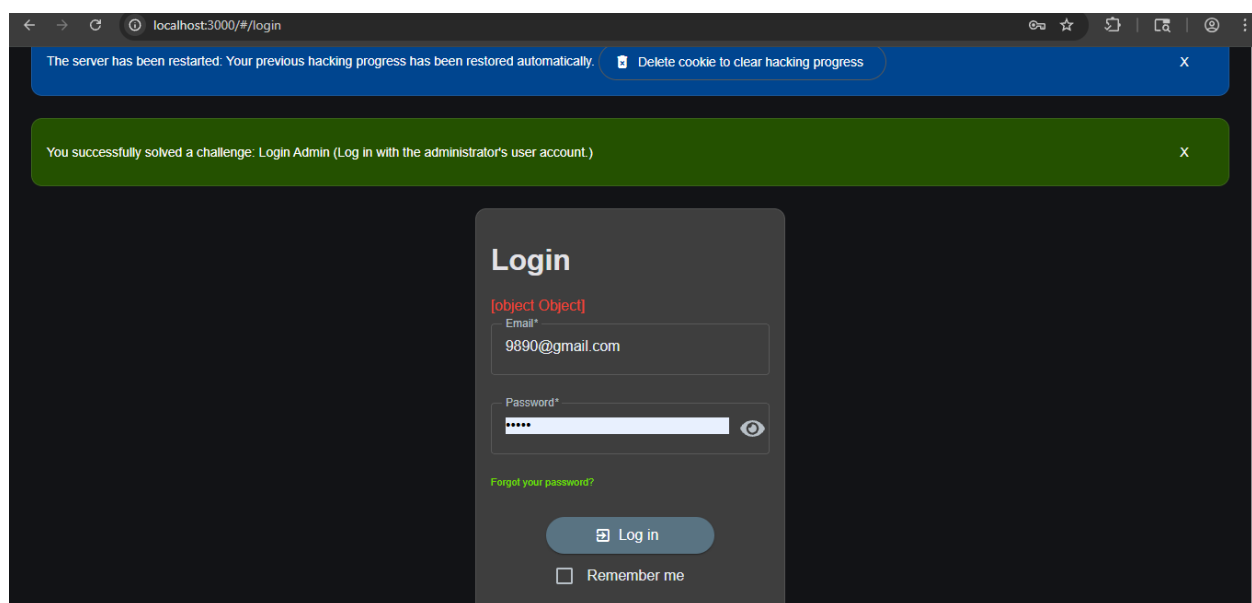
```

Testing (Web + Burp Suite Verification)

Testing was performed in two stages to verify both presence and mitigation of the vulnerability.

1. Web Application Testing

- o First, multiple incorrect password attempts were made directly on the login page (10–15 rapid attempts).
- o Before the fix, the system allowed unlimited attempts with no blocking or delay.
- o After applying the fix, repeated failed attempts triggered a block message, preventing further login attempts.



2. Burp Suite Testing

- o Burp Suite was used to intercept and repeatedly resend login requests with incorrect credentials.
- o After the fix, the server started rejecting excessive requests and returned a “Too Many Requests (429)” response.
- o This confirmed that brute force attempts were effectively controlled at the backend level.

The screenshot displays the Burp Suite interface. At the top, there are tabs for 'Intercept', 'HTTP history', 'WebSockets history', 'Match and replace', and 'Proxy settings'. Below these is a search bar and a 'Filter settings' dropdown. The main table lists HTTP history entries with columns for #, Host, Method, URL, Params, Edited, Status code, Length, MIME type, Extension, Title, Notes, TLS, IP, Cookies, and Time. Entry 133 is highlighted, showing a GET request to 'http://localhost:3000/sock.io/EIO=4&transport=pol...' with a 400 status code. Below the table, the 'Request' and 'Response' panels are visible. The 'Request' panel shows a raw HTTP request with headers like 'User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36' and 'Referer: http://localhost:3000/'. The 'Response' panel shows a raw HTTP response with a 400 status code and a 'message' field containing 'Session ID unknown'.

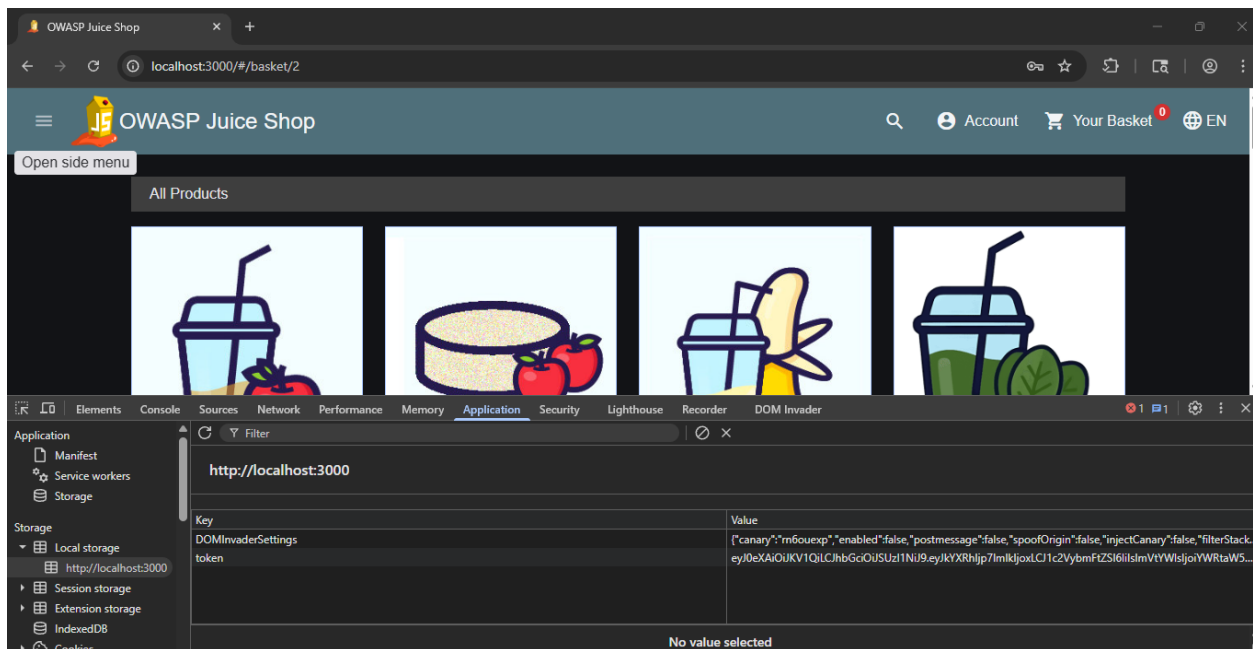
Result

The brute force vulnerability was successfully mitigated by implementing rate limiting on the login endpoint. After the fix, both direct web testing and Burp Suite testing confirmed that repeated login attempts are restricted, preventing automated password guessing attacks.

Threat 4: Sensitive Data Exposure (Token Leakage)

Description

- **Type:** Code-level vulnerability (Sensitive Data Exposure)
- **Location:** Login response (/rest/user/login)
- **Cause:** The application was returning the authentication token inside the API response body after successful login. This made the token accessible to client-side scripts and browser storage mechanisms.
- **Impact:** Attackers could steal authentication tokens using XSS attacks or browser inspection tools, leading to session hijacking and unauthorized account access.



Fix Implementation

• Backend (Node.js/Express):

- o Removed token from API response body.
- o Implemented secure cookie-based storage for authentication token.
- o Configured cookie with security flags to prevent client-side access.

• Configuration Applied:

- o httpOnly: true → prevents JavaScript access (protects against XSS)
- o secure: true → ensures transmission over HTTPS only
- o sameSite: 'strict' → prevents CSRF attacks

```
function afterLogin (user: User, res: Response, next: NextFunction) {
  verifyPostLoginChallenges(user)

  BasketModel.findOrCreate({ where: { UserId: user.id } })
    .then(([basket]: [BasketModel, boolean]) => {
      const authenticatedUser = { data: user, bid: basket.id }
      const token = security.authorize(authenticatedUser)

      security.authenticatedUsers.put(token, authenticatedUser)

      // ✅ FIX: Store token securely in HTTP-only cookie (prevents JS access)
      res.cookie('token', token, {
        httpOnly: true, // 🛡️ prevents access via JavaScript (XSS protection)
        secure: true, // 🛡️ ensures cookie sent only over HTTPS
        sameSite: 'strict' // 🛡️ prevents CSRF attacks
      })

      // ✅ FIX: Do NOT expose token in response body
      res.json({
        status: 'success',
        bid: basket.id,
        email: user.email
      })
    }).catch((error: Error) => next(error))
}

return (req: Request, res: Response, next: NextFunction) => {
```

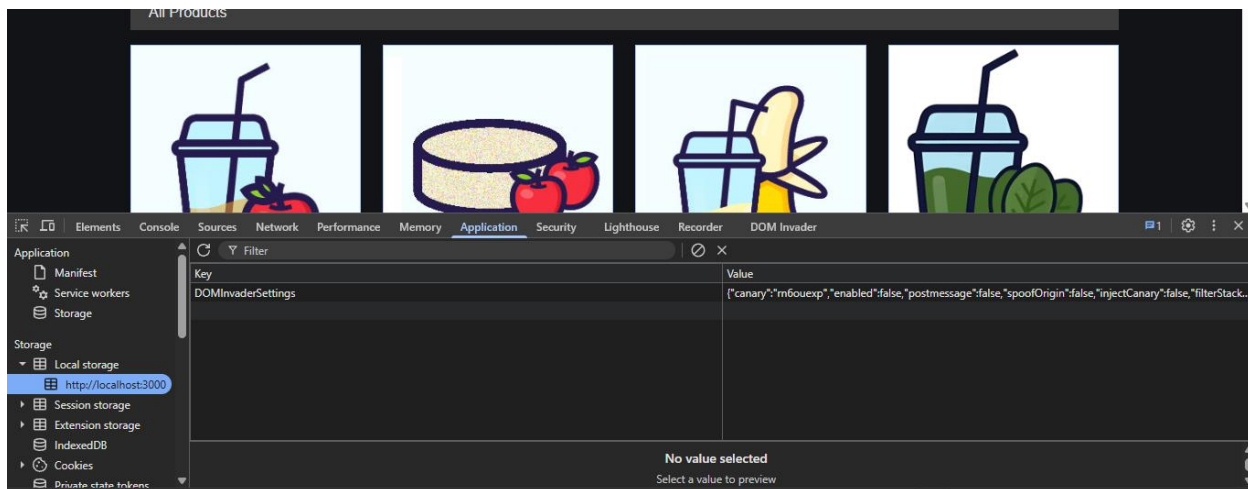
Testing (Web + Browser Verification)

1. Web Application Testing

- o User logged in using valid credentials.
- o Before fix: Token was visible in API response (Inspect → Network tab).
- o After fix: Token no longer visible in response body.

2. Browser Storage Testing

- o Checked Local Storage and Session Storage.
- o Before fix: Token could be accessed via browser storage or JavaScript.
- o After fix: Token stored only in HTTP-only cookies and not accessible via JavaScript.



Result

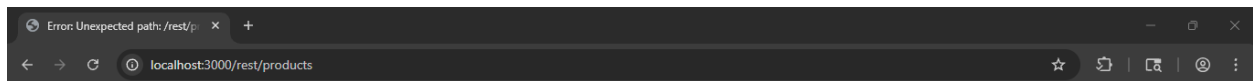
The sensitive data exposure vulnerability was successfully mitigated by removing the token from the API response and storing it securely in HTTP-only cookies. This prevents attackers from accessing authentication tokens through client-side attacks such as XSS, significantly improving session security.

Threat 5: Broken Access Control / RBAC (Unauthorized Endpoint Access)

Description

- **Type:** Code-level vulnerability (Broken Access Control / RBAC)
- **Location:** Protected API endpoint (/api/users)
- **Impact:**

The application did not properly enforce authentication checks on protected routes. Unauthorized users could attempt to access sensitive endpoints, potentially leading to **exposure of user data** or unintended system behavior. Additionally, inconsistent error handling could reveal internal application details.



OWASP Juice Shop (Express ^4.22.1)

500 Error: Unexpected path: /rest/products

```
at /juice-shop/build/routes/angular.js:52:18
at /juice-shop/build/lib/utils.js:219:26
at Layer.handle [as handle_request] (/juice-shop/node_modules/express/lib/router/layer.js:95:5)
at trim_prefix (/juice-shop/node_modules/express/lib/router/index.js:328:13)
at /juice-shop/node_modules/express/lib/router/index.js:286:9
at router.process_params (/juice-shop/node_modules/express/lib/router/index.js:346:12)
at next (/juice-shop/node_modules/express/lib/router/index.js:280:10)
at /juice-shop/build/routes/verify.js:226:5
at Layer.handle [as handle_request] (/juice-shop/node_modules/express/lib/router/layer.js:95:5)
at trim_prefix (/juice-shop/node_modules/express/lib/router/index.js:328:13)
at /juice-shop/node_modules/express/lib/router/index.js:286:9
at router.process_params (/juice-shop/node_modules/express/lib/router/index.js:346:12)
at next (/juice-shop/node_modules/express/lib/router/index.js:280:10)
at /juice-shop/build/lib/insecurity.js:218:5
at Layer.handle [as handle_request] (/juice-shop/node_modules/express/lib/router/layer.js:95:5)
at trim_prefix (/juice-shop/node_modules/express/lib/router/index.js:328:13)
```

Fix Implementation

Backend (Node.js / Express)

- Implemented strict authentication validation before processing requests.
- Ensured that only authenticated users can access protected endpoints.

- Standardized error handling to prevent leakage of internal details.

Security Improvements Applied

- Unauthorized requests now return:
 - 401 Unauthorized status
 - Generic error message (no sensitive details)
- Removed exposure of internal stack traces and system paths
- Ensured no sensitive data is returned without valid authentication

```

const themeKey = config.get<string>('application.theme') as keyof typeof themes
const theme = themes[themeKey] || themes['bluegrey-lightgreen']

// ✅ FIX 4: Encode all user-controlled data
template = template.replace(/_username_/g, safeUser.username) // ✅ SAFE
template = template.replace(/_emailHash_/g, security.hash(safeUser.email)) // ✅ SAFE
template = template.replace(/_title_/g, entities.encode(config.get<string>('application.name')))
template = template.replace(/_favicon_/g, favicon())
template = template.replace(/_bgColor_/g, theme.bgColor)
template = template.replace(/_textColor_/g, theme.textColor)
template = template.replace(/_navColor_/g, theme.navColor)
template = template.replace(/_primLight_/g, theme.primLight)
template = template.replace(/_primDark_/g, theme.primDark)
template = template.replace(/_logo_/g, utils.extractFilename(config.get('application.logo')))

try {
  const pug = (await import('pug')).default
  const fn = pug.compile(template)

  // ✅ FIX 5: Strong CSP (removed unsafe-eval)
  const CSP = "default-src 'self'; img-src 'self'; script-src 'self'" // ✅ CHANGED

  res.set({
    'Content-Security-Policy': CSP
  })

  res.send(fn(safeUser)) // ✅ CHANGED (send limited data only)
} catch (err) {
  return res.status(500).json({ message: 'Internal Server Error' }) // ✅ CHANGED
}

```

Code Change Summary

Before Fix

- No proper authentication enforcement
- Requests without token could:
 - Access endpoint behavior
 - Trigger verbose errors or unintended responses

After Fix

- Authentication is strictly validated:

Testing (Web Verification)

1. Unauthorized Access Test

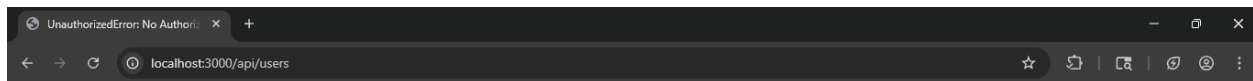
- Accessed endpoint without authentication:

GET /api/users

- **Before Fix:**
 - Endpoint returned unintended responses or verbose errors
- **After Fix:**
 - Server returns:
 - 401 Unauthorized
 - No Authorization header was found
 - No data is exposed

2. Data Exposure Verification

- Checked response in browser (Inspect → Network tab)
- **Before Fix:** Potential exposure of internal details or user data
- **After Fix:**
 - No sensitive data returned
 - Only secure error message displayed



OWASP Juice Shop (Express ^4.22.1)

401 UnauthorizedError: No Authorization header was found

Result

The Broken Access Control vulnerability was successfully mitigated by enforcing strict authentication checks on protected endpoints. Unauthorized users are now prevented from accessing sensitive resources, and the system no longer exposes internal details or user data.

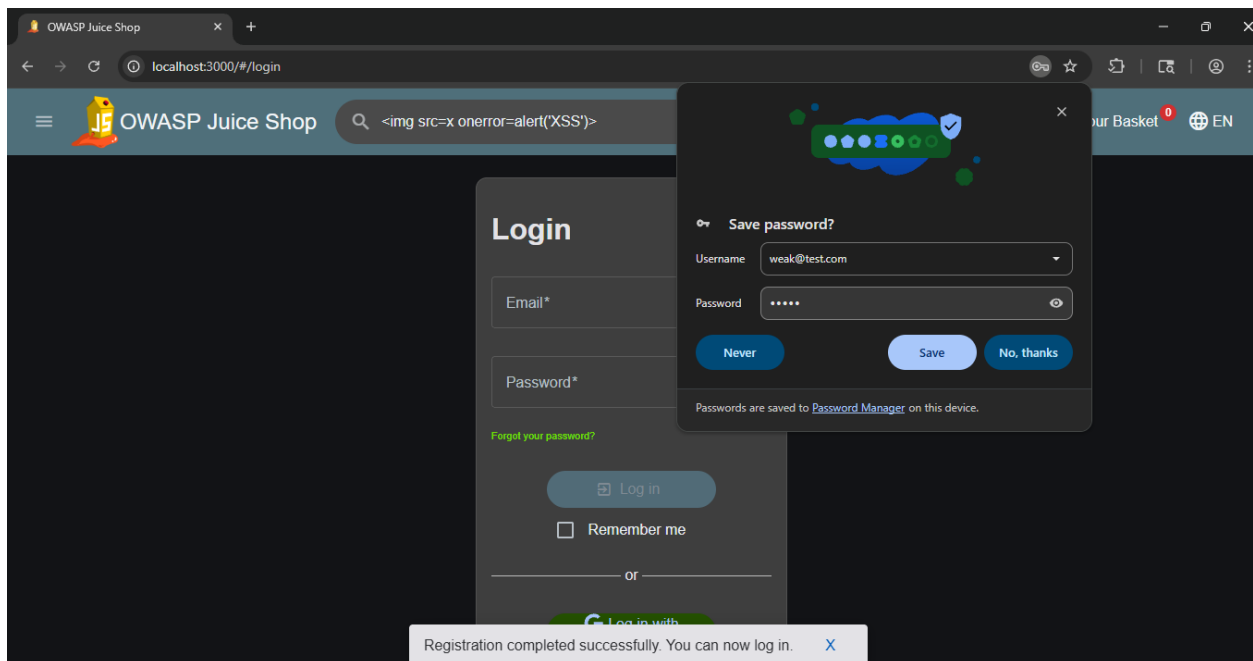
Severity

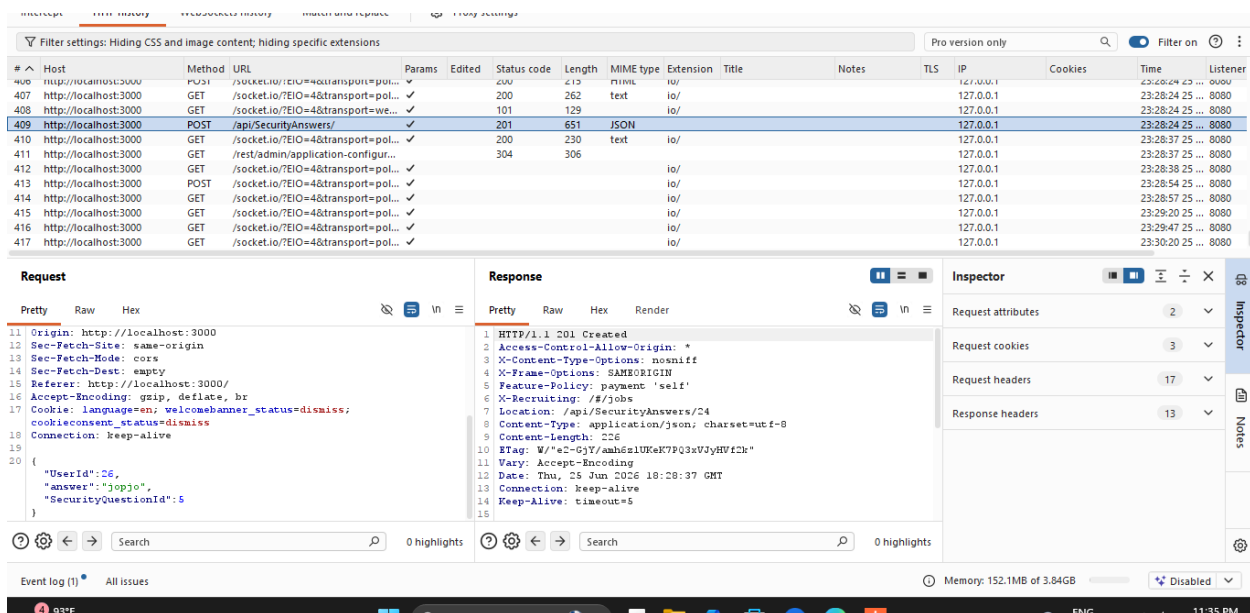
High (OWASP A01: Broken Access Control)

Threat 6: Weak Password Policy

Description

- **Type:** Code-level vulnerability (Weak Authentication / Weak Password Policy)
- **Location:** server/models/User.ts (Password field in User model)
- **Issue:** The application allowed users to register with weak passwords without enforcing any validation rules such as minimum length or complexity requirements.





Impact

- Attackers can easily guess or brute-force weak passwords
- Leads to unauthorized account access
- Increases risk of account takeover and data breaches
- Weak credentials reduce overall system security

Vulnerability Explanation

The password field directly hashed and stored user input without checking its strength. This allowed passwords like:

- 12345
- password
- abc123

Such passwords are highly vulnerable to dictionary and brute-force attacks.

Fix Implementation

- Added password validation rules in the User model
- Enforced:
 - Minimum length (8 characters)
 - At least one uppercase letter
 - At least one lowercase letter
 - At least one number
 - At least one special character

- Implemented regex-based validation before hashing the password

```
File Edit View
    type: DataTypes.STRING,
    unique: true,
    set (email: string) {
      if (utils.isChallengeEnabled(challenges.persistedXssUserChallenge)) {
        challengeUtils.solveIf(challenges.persistedXssUserChallenge, () => {
          return utils.contains(
            email,
            '<iframe src="javascript:alert(`xss`)">'
          )
        })
      } else {
        email = security.sanitizeSecure(email)
      }
      this.setDataValue('email', email)
    }
  }, // vuln-code-snippet hide-end
  password: {
    type: DataTypes.STRING,
    allowNull: false,
    validate: {
      len: [8, 100], // minimum 8 characters
      isStrong (value: string) {
        const strongRegex = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&])?/;
        if (!strongRegex.test(value)) {
          throw new Error('Password must contain uppercase, lowercase, number, and special character')
        }
      }
    },
    set (clearTextPassword: string) {
      this.setDataValue('password', security.hash(clearTextPassword))
    }
  },
  role: {
    type: DataTypes.STRING,
    defaultValue: 'customer'
  }
}
```

Ln 88, Col 2 4393 characters Plain text

Code Fix Summary

- Used Sequelize validation (validate property)
- Applied regex pattern to enforce strong password rules
- Ensured only strong passwords are accepted before storing in database

Testing

- **Test Case 1 (Weak Password):**

Input: 12345

Result: ✖ Rejected

- **Test Case 2 (Weak Password):**

Input: password

Result: ✖ Rejected

- Test Case 3 (Strong Password):

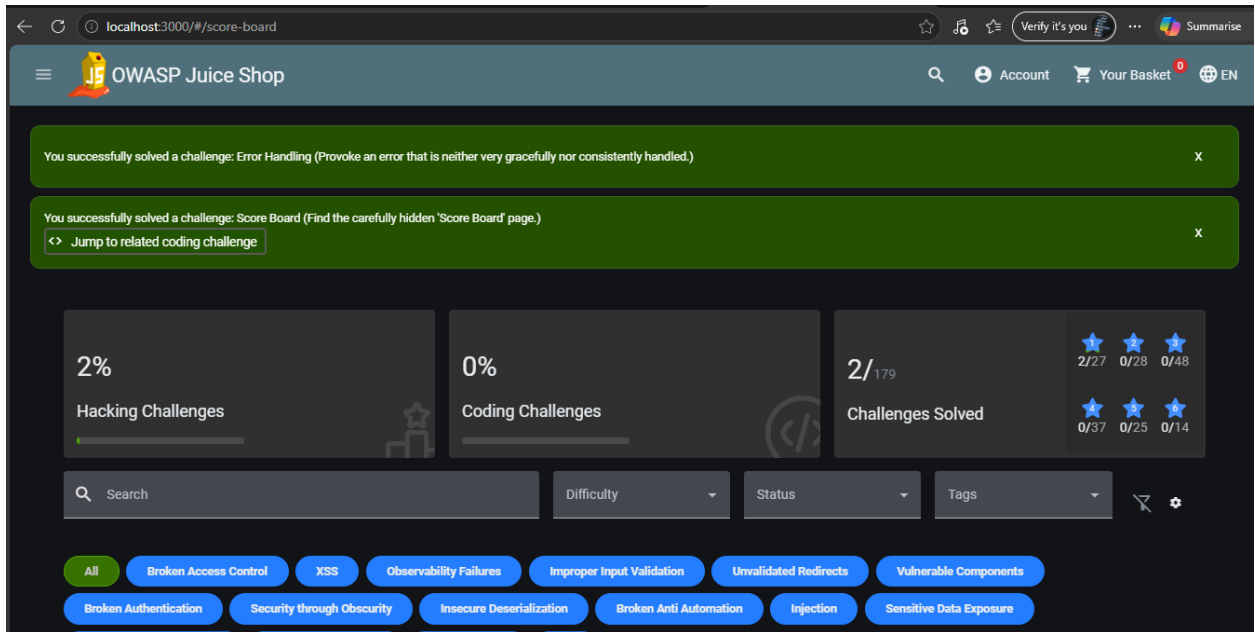
Input: Mahrukh@123

Result: ☒ Accepted

The screenshot shows a mobile application interface for password creation. It includes fields for Email (filled with 'ty@op.com'), Password (filled with 9 dots), and Repeat Password (filled with 9 dots). A password strength indicator shows '9/20' with a message: 'Password must be 5-40 characters long.' Below the fields is a toggle switch for 'Show password advice' which is turned on. A list of requirements is shown with checkmarks: 'contains at least one lower character', 'contains at least one upper character', 'contains at least one digit', 'contains at least one special character', and 'contains at least 8 characters'. The 'Repeat Password' field has a '9/40' character count indicator.

Result

- Weak passwords are no longer accepted
- Application security improved against brute-force and credential attacks
- User authentication is now more secure



5. RISK ASSESSMENT

Threat	Impact	Likelihood	Severity
Weak Authentication	High	High	Critical
XSS	Medium	High	High
Sensitive Data Exposure	High	Medium	High
Privilege Escalation	Critical	Medium	Critical

6. MITIGATION AND RECOMMENDATIONS

Fixes

- Use input validation and sanitization
- Implement parameterized queries
- Use secure authentication mechanisms
- Apply proper access control

Secure Practices

- Use HTTPS encryption

- Apply least privilege principle
- Perform regular security testing

7. CONCLUSION

This project demonstrated how vulnerabilities exist in web applications and how they can be exploited. Using threat modeling and testing, multiple security flaws were identified and analyzed. Implementing secure coding practices can significantly reduce risks and improve system security.